

Logika v Računalništvu project

Dated 2026-05-19
Deadline 2026-06-05

Problem 1 (*). Define a type of formulas called `Formula`, with the following grammar:

$$\begin{array}{lcl} \text{Formula} & \rightarrow & \text{Var } n \\ & | & \neg \text{Formula} \\ & | & \text{Formula} \wedge \text{Formula} \\ & | & \text{Formula} \vee \text{Formula} \end{array}$$

Problem 2 (*). Define a type of negation normal form formulas called `NNF`, with the following grammar:

$$\begin{array}{lcl} \text{Literal} & \rightarrow & \text{Var } n \\ & | & \neg \text{Var } n \\ \text{NNF} & \rightarrow & \text{Literal} \\ & | & \text{NNF} \wedge \text{NNF} \\ & | & \text{NNF} \vee \text{NNF} \end{array}$$

Problem 3 (*). Construct a function `to-nnf` of type `Formula → NNF` that converts a formula to an equivalent formula in negation normal form.

Note: You may find a more suitable name for the functions.

Problem 4 (**). Copy the `Assoc` module from week 9 exercises and complete it to a fully working implementation of an associative structure you want (associative list, dictionary, etc.). For example, use the `NoDup` structure from the exercises.

Then use

```
open Assoc ℕ test ≡ Bool
Assignment : Set
Assignment = Assoc
```

Note: it is also OK to specialise the definitions to $K = \mathbb{N}$ and $V = \text{Bool}$, for example to use `Fin n`. You are also free to use the `stdlib` definitions like

```
open import Relation.Binary using (Decidable; DecidableEquality)
open import Data.List.Relation.Unary.Any using (Any; any?)
open import Data.List.Relation.Unary.All using (All; all?)
open import Relation.Nullary using (yes; no; ¬_; ¬?)
```

Problem 5 (*). Define an evaluation function `eval : Assignment → Formula → Maybe Bool` assigning to each assignment of variables and formula its truth value.

Problem 6 (*). Define an evaluation function `eval-nnf : Assignment → NNF → Maybe Bool` assigning to each assignment of variables and negation normal from formula its truth value.

Problem 7 (*). Define a type of conjunction normal form formulas called `CNF`, with the following grammar:

<code>Literal</code>	$\rightarrow$	<code>Var n</code>
	$ $	<code>$\neg$Var n</code>
<code>Disjunct</code>	$\rightarrow$	<code>Literal</code>
	$ $	<code>Literal $\vee$ Disjunct</code>
<code>CNF</code>	$\rightarrow$	<code>Disjunct</code>
	$ $	<code>Disjunct $\wedge$ CNF</code>

Problem 8 (*). Define an evaluation function `eval-cnf : Assignment → CNF → Maybe Bool` assigning to each assignment of variables and conjunction normal from formula its truth value.

Problem 9 (/***)**. Write an SAT solver for CNF formulas. It should output either an assignment such that evaluating the formula at that assignment evaluates to `true` or that no such assignment exists.

Note: a more complex implementation (e. g. DPLL) will be graded higher.

Problem 10 ()**. Show that the SAT solver you implemented is indeed correct, if that is not obvious from the output type of the SAT solver.

Problem 11 (/***)**. Write a function that converts an NNF formula to an equisatisfiable CNF formula.

Note: it is intended for you to attempt to implement the Tseytin transformation. A simpler implementation will be accepted for partial credit.

Problem 12 (*). To tie the bow on the whole thing, use the above to construct a SAT solver for any formula.